

CLOVER App Summary

What it is

- CLOVER is a GPU-native exact k-nearest-neighbor (kNN) research codebase accompanying the paper 'CLOVER: A GPU-native, Spatio-graph-based Approach to Exact kNN'.
- It benchmarks CUDA-based kNN strategies (linear-scan + spatio-graph variants) via one executable: linear-scans.

Who it's for

- Primary persona: GPU systems/ML researchers and performance engineers evaluating exact kNN implementations and CUDA-kernel optimizations.

What it does

- Builds a CUDA/C++ executable (linear-scans) from src/*.cu and include/*.cuh.
- Supports CLI-selected algorithms: Bitonic, Warpwise, Hubs variants, FAISS variants, and Treelogy KD-tree.
- Runs synthetic experiments with fixed-seed random 3D data.
- Runs mesh experiments by scanning ../meshes and loading .txt point files.
- Copies points/queries to GPU, dispatches selected kernels, returns neighbors+distances.
- Prints timing tuples in release mode and can print outputs for inspection.
- Optionally enables FAISS-backed paths via build flag USE_FAISS/LINK_FAISS.

How it works (architecture)

- Build layer: CMake compiles CUDA sources, links cuBLAS, conditionally links FAISS/OpenMP.
- Orchestration: linear-scans.cu parses algorithm index, prepares data, and loops benchmark sizes.
- Data flow: host arrays/vectors -> cudaMemcpy to device -> algorithm kernel -> host vectors -> stdout.
- Algorithm layer: include/ provides bitonic, warpwise, hubs, FAISS wrappers, and Treelogy integration.
- External services/components: Not found in repo.

How to run (minimal)

- 1) Install CUDA+GCC (README cites CUDA 12.6, GCC 13.3 for local Linux).
- 2) mkdir build && cd build
- 3) cmake -DCMAKE_BUILD_TYPE=Release ..
- 4) make
- 5) ./linear-scans 2 (or another algorithm index)